



CHECKLIST DE VERIFICACIÓN - Sprint 1



Verificación Post-Instalación

Usa este checklist para asegurar que el sistema está correctamente instalado y funcionando.

1 INSTALACIÓN BÁSICA

Entorno Python

- ☐ Python 3.8+ instalado
- ☐ Entorno virtual creado (venv/)
- ☐ Entorno virtual activado
- ☐ Dependencias instaladas (pip install -r requirements.txt)

Base de Datos

- ☐ MySQL 5.7.8+ instalado
- ☐ MySQL server corriendo
- ☐ Base de datos tea_management creada
- ☐ Usuario MySQL con permisos adecuados

Configuración

- ☐ Archivo .env creado desde .env.example
 - ☐ DB_NAME configurado en .env
 - ☐ DB_USER configurado en .env
 - ☐ DB_PASSWORD configurado en .env
 - ☐ SECRET_KEY generado y configurado
 - ☐ Directorio logs/ existe
 - ☐ Directorio media/ existe
 - ☐ Directorio static/ existe
-

2 MIGRACIONES Y DATOS

Django Setup

- ☐ Migraciones creadas (python manage.py makemigrations)
- ☐ Migraciones aplicadas (python manage.py migrate)
- ☐ Superusuario creado (python manage.py createsuperuser)
- ☐ Sin errores en migraciones

Verificar Tablas



sql

```
-- Ejecuta en MySQL para verificar
USE tea_management;
SHOW TABLES;

-- Deberías ver:
-- usuarios_usuario
-- usuarios_perfil
-- usuarios_registroacceso
-- + tablas de Django (auth, sessions, etc.)
```

- ☐ Tabla usuarios_usuario existe
- ☐ Tabla usuarios_perfil existe
- ☐ Tabla usuarios_registroacceso existe
- ☐ Tablas de Django admin existen

3 SERVIDOR Y ACCESO

Servidor de Desarrollo

- ☐ Servidor inicia sin errores (python manage.py runserver)
- ☐ Puerto 8000 accesible
- ☐ No hay warnings de configuración críticos

URLs Principales

- ☐ <http://127.0.0.1:8000/> responde
- ☐ <http://127.0.0.1:8000/login/> carga
- ☐ <http://127.0.0.1:8000/admin/> carga
- ☐ <http://127.0.0.1:8000/api/docs/> carga (Swagger)
- ☐ <http://127.0.0.1:8000/api/redoc/> carga (ReDoc)

Login Admin

- ☐ Login en admin exitoso con superusuario
- ☐ Panel de admin visible
- ☐ Sección "USUARIOS" visible en admin
- ☐ Sección "GESTIÓN DE USUARIOS" visible

4 FUNCIONALIDAD CORE

Gestión de Usuarios (Admin)

- ☐ Puede crear usuario desde admin
- ☐ Puede ver lista de usuarios
- ☐ Puede editar usuario existente
- ☐ Perfil se crea automáticamente con usuario
- ☐ Puede ver perfil inline en usuario
- ☐ Puede editar perfil desde usuario
- ☐ Puede filtrar usuarios por rol
- ☐ Puede filtrar usuarios por estado
- ☐ Puede buscar usuarios

Autenticación

- ☐ Login exitoso en /login/
- ☐ Redirect a dashboard después de login
- ☐ Logout funciona correctamente
- ☐ Registro de acceso se crea automáticamente
- ☐ Sesión persiste entre recargas

API REST

- ☐ Puede obtener token JWT (POST /api/token/)
 - ☐ Token es válido y tiene estructura correcta
 - ☐ Puede listar usuarios con token (GET /api/usuarios/)
 - ☐ Respuesta tiene paginación
 - ☐ Puede filtrar por rol en API
 - ☐ Puede buscar usuarios en API
 - ☐ Documentación Swagger funciona
 - ☐ Puede probar endpoints desde Swagger
-

5 TESTING

Ejecutar Tests

- ☐ python manage.py test ejecuta sin errores
- ☐ Todos los tests pasan (30+ tests)
- ☐ No hay warnings críticos en tests
- ☐ Tests de modelos pasan
- ☐ Tests de vistas pasan
- ☐ Tests de API pasan

Tests Específicos



bash

Ejecuta estos comandos y verifica que pasen

python manage.py test apps.usuarios.tests.UsuarioModelTest

 Debería pasar todos los tests de modelo Usuario

python manage.py test apps.usuarios.tests.PerfilModelTest

 Debería pasar todos los tests de modelo Perfil

python manage.py test apps.usuarios.tests.UsuarioViewsTest

 Debería pasar todos los tests de vistas

python manage.py test apps.usuarios.tests.UsuarioAPITest

 Debería pasar todos los tests de API

- ☐ Tests de Usuario pasan
- ☐ Tests de Perfil pasan
- ☐ Tests de Vistas pasan
- ☐ Tests de API pasan

6 MODELOS Y DATOS

Crear Usuario de Prueba



python

Ejecuta en python manage.py shell

```
from apps.usuarios.models import Usuario
```

Crear terapeuta

```
terapeuta = Usuario.objects.create_user(  
    username='test_terapeuta',  
    email='terapeuta@test.com',  
    password='Test123!',  
    first_name='Test',  
    last_name='Terapeuta',  
    rol='TERAPEUTA'  
)
```

Verificar que el perfil se creó automáticamente

```
print(terapeuta.perfil) # Debería mostrar el perfil
```

Verificar propiedades

```
print(terapeuta.es_terapeuta) # Debería ser True
```

```
print(terapeuta.es_admin) # Debería ser False
```

- ☐ Usuario se crea correctamente
- ☐ Perfil se crea automáticamente
- ☐ Propiedades funcionan correctamente
- ☐ Rol se asigna correctamente

Verificar Relaciones



python

En python manage.py shell

```
from apps.usuarios.models import Usuario, Perfil
```

Obtener usuario

```
usuario = Usuario.objects.first()
```

Verificar relación one-to-one con perfil

```
perfil = usuario.perfil
```

```
print(perfil.usuario == usuario) # Debería ser True
```

Agregar certificación

```
perfil.agregar_certificacion(
```

```
    nombre='Test Cert',
```

```
    institucion='Test Uni',
```

```
    fecha='2024-01-01'
```

```
)
```

Verificar que se agregó

```
print(len(perfil.certificaciones)) # Debería ser >= 1
```

- ☐ Relación Usuario-Perfil funciona
- ☐ Método agregar_certificacion funciona
- ☐ JSONField almacena datos correctamente

7 API ENDPOINTS

Test Manual de API



bash

1. Obtener token

```
curl -X POST http://127.0.0.1:8000/api/token/ \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"tu_password"}'
```

Debería retornar:

```
# {"access":"token...", "refresh":"token..."}
```

2. Listar usuarios (reemplaza TOKEN con tu token)

```
curl -X GET http://127.0.0.1:8000/api/usuarios/ \  
-H "Authorization: Bearer TOKEN"
```

Debería retornar lista de usuarios con paginación

3. Ver usuario actual

```
curl -X GET http://127.0.0.1:8000/api/usuarios/me/ \  
-H "Authorization: Bearer TOKEN"
```

Debería retornar tu usuario

- ☐ Token se obtiene correctamente
- ☐ Listar usuarios funciona con token
- ☐ Endpoint /me/ retorna usuario correcto
- ☐ Respuestas tienen formato JSON correcto

8 SEGURIDAD

Configuraciones de Seguridad

- ☐ SECRET_KEY es única y no está en control de versiones
- ☐ .env no está en Git (verificar .gitignore)
- ☐ Contraseñas se hashean con Argon2
- ☐ CSRF protection está habilitado
- ☐ DEBUG = False en producción (cuando aplique)

Permisos

- ☐ Usuario no autenticado no puede acceder a dashboard
 - ☐ Usuario normal no puede acceder a admin
 - ☐ API requiere autenticación
 - ☐ Solo admin puede crear usuarios
 - ☐ Usuarios solo pueden editar su propio perfil
-

9 DOCUMENTACIÓN

Archivos de Documentación

- ☐ README.md existe y es completo
- ☐ SPRINT_1_REVIEW.md detalla todo lo implementado
- ☐ INICIO_RAPIDO.md tiene instrucciones claras
- ☐ RESUMEN_EJECUTIVO.md resume el proyecto
- ☐ INDICE_ARCHIVOS.md lista todos los archivos
- ☐ Este CHECKLIST.md existe

Código Documentado

- ☐ Models tienen docstrings
 - ☐ Views tienen docstrings
 - ☐ API serializers tienen docstrings
 - ☐ Funciones complejas tienen comentarios
 - ☐ Tests tienen descripciones claras
-

10 PREPARACIÓN PARA SPRINT 2

Estructura Preparada

- ☐ Directorio apps/consultorios/ existe
- ☐ Directorio apps/terapias/ existe
- ☐ Directorio apps/procedimientos/ existe
- ☐ Archivos __init__.py creados en cada app
- ☐ Directorio templates/ existe
- ☐ Directorio static/ con subdirectorios existe

Git

- ☐ Repositorio Git inicializado (opcional)
 - ☐ .gitignore configurado correctamente
 - ☐ Commit inicial realizado (opcional)
-

RESUMEN DE VERIFICACIÓN

Crítico (Debe estar)

- ☐ Servidor inicia sin errores
- ☐ Admin funciona
- ☐ Puede crear usuarios
- ☐ Tests pasan
- ☐ API responde

Importante (Debería estar)

- ☐ JWT tokens funcionan
- ☐ Perfil se crea automáticamente
- ☐ Registros de acceso se crean
- ☐ Permisos funcionan correctamente

Recomendado (Bueno tener)

- ☐ Documentación leída
- ☐ Usuarios de prueba creados
- ☐ API probada manualmente
- ☐ Swagger explorado

 SOLUCIÓN DE PROBLEMAS

Si algo no funciona:

Problema: Servidor no inicia



bash

1. Verificar que estás en el entorno virtual
`which python` *# Debería apuntar a venv/*

2. Verificar configuración
`python manage.py check`

3. Ver errores específicos
`python manage.py runserver --traceback`

Problema: Tests fallan



bash

1. Verificar base de datos

```
python manage.py dbshell
```

2. Recrear base de datos de test

```
python manage.py test --keepdb
```

3. Ejecutar test específico

```
python manage.py test apps.usuarios.tests.UsuarioModelTest.test_crear_usuario_valido -v 2
```

Problema: API no responde



bash

1. Verificar que DRF está instalado

```
pip show djangorestframework
```

2. Verificar URLs

```
python manage.py show_urls | grep api
```

3. Verificar token

```
curl -X POST http://127.0.0.1:8000/api/token/ \
-H "Content-Type: application/json" \
-d '{"username": "admin", "password": "admin"}'
```

MÉTRICAS DE ÉXITO

Sprint 1 está completo cuando:

- ☒ 100% de los checkboxes críticos marcados
- ☒ 90%+ de los checkboxes importantes marcados
- ☒ 80%+ de todos los checkboxes marcados
- ☒ Todos los tests pasan
- ☒ Servidor funciona sin errores

SIGUIENTE PASO

Una vez completado este checklist:

1. Marca la fecha de finalización: _____

- 2. **Revisa SPRINT_1_REVIEW.md** para detalles completos
- 3. **Lee sobre Sprint 2** en SPRINT_1_REVIEW.md
- 4. **Planifica Sprint 2:** Consultorios y Espacios

 AYUDA ADICIONAL

Si encuentras problemas:

- 1. Revisa README.md - Sección de Troubleshooting
- 2. Revisa INICIO_RAPIDO.md - Sección de Problemas Comunes
- 3. Ejecuta `python manage.py check` para diagnóstico
- 4. Revisa los logs en `logs/django.log`

Estado del Sprint 1:

- ☐ En Progreso
- ☐ Bloqueado
- ☐ Completo 

Fecha de Verificación: _____

Verificado por: _____

Observaciones:

¡Éxito en tu proyecto! 